시리얼 통신(SPI & I2C)

모듈 RTL 설계 및 검증

SystemVerilog를 활용한 Master/Slave 통신 아키텍처 구현

김민수

2026-04-20

목차

1. 프로젝트 개요

2. SPI 설계 및 검증

3. I2C 설계 및 검증

4. 결론 (Trouble shooting 및 느낀 점)

프로젝트 개요 (Project Overview)

1. 프로젝트 목표

- 대표적인 동기식 직렬 통신인 SPI와 I2C 프로토콜의 하드웨어(RTL) 직접 구현
- SystemVerilog를 이용한 Master 및 Slave FSM 설계
- FPGA Board-to-Board 환경에서의 데이터 송수신 검증

2. 개발 및 검증 환경

- 개발 언어: SystemVerilog
- 타겟 디바이스: FPGA 보드 2대 (Master Board ↔ Slave Board 구조)
- RTL
  - SPI / I2C Master 모듈
  - SPI / I2C Slave 모듈
  - FND(7-Segment) Controller 및 스위치 I/O

이론적 배경 (SPI vs I2C 비교)

구분	I2C (Inter-Integrated Circuit)	SPI (Serial Peripheral Interface)
통신 방향	반이중 (Half-Duplex)	전이중 (Full-Duplex)
핀(선) 개수	2개 (SCL, SDA)	4개 (SCLK, MOSI, MISO, CS_n)
전송 속도	비교적 느림 (Standard 100kHz ~ Fast 400kHz)	매우 빠름 (수 MHz ~ 수십 MHz 이상)
하드웨어 특성	Pull-up 저항 및 Open-Drain 제어 로직 필요	Push-Pull 구조, 단순하고 직관적인 인터페이스

SPI 통신 프로토콜 및 모드 (Modes)

SPI는 클럭의 극성(CPOL)과 위상(CPHA) 설정에 따라 4가지 모드로 나뉘며 SPI Master 모듈은 4가지 모드를 모두 지원하도록 설계

모드 (Mode)	CPOL (Clock Polarity)	CPHA (Clock Phase)	데이터 캡처 시점 (Sample)
Mode 0	0 (IDLE 시 Low)	0	첫 번째 엣지 (Rising Edge)
Mode 1	0 (IDLE 시 Low)	1	두 번째 엣지 (Falling Edge)
Mode 2	1 (IDLE 시 High)	0	첫 번째 엣지 (Falling Edge)
Mode 3	1 (IDLE 시 High)	1	두 번째 엣지 (Rising Edge)

SPI Master 모듈 설계 아키텍처

IDLE / START

시작 신호가 인가되면 CS_n 신호를 Low로 활성화하고 통신을 준비합. CPOL 값에 따라 SCLK의 논리 레벨이 결정.

DATA 송수신

분주된 half_tick 단위로 SCLK를 토글시키며 선택된 CPHA 위상에 정확히 맞추어 MOSI로 시프트 아웃하고 MISO 데이터를 가져옴

STOP 조건

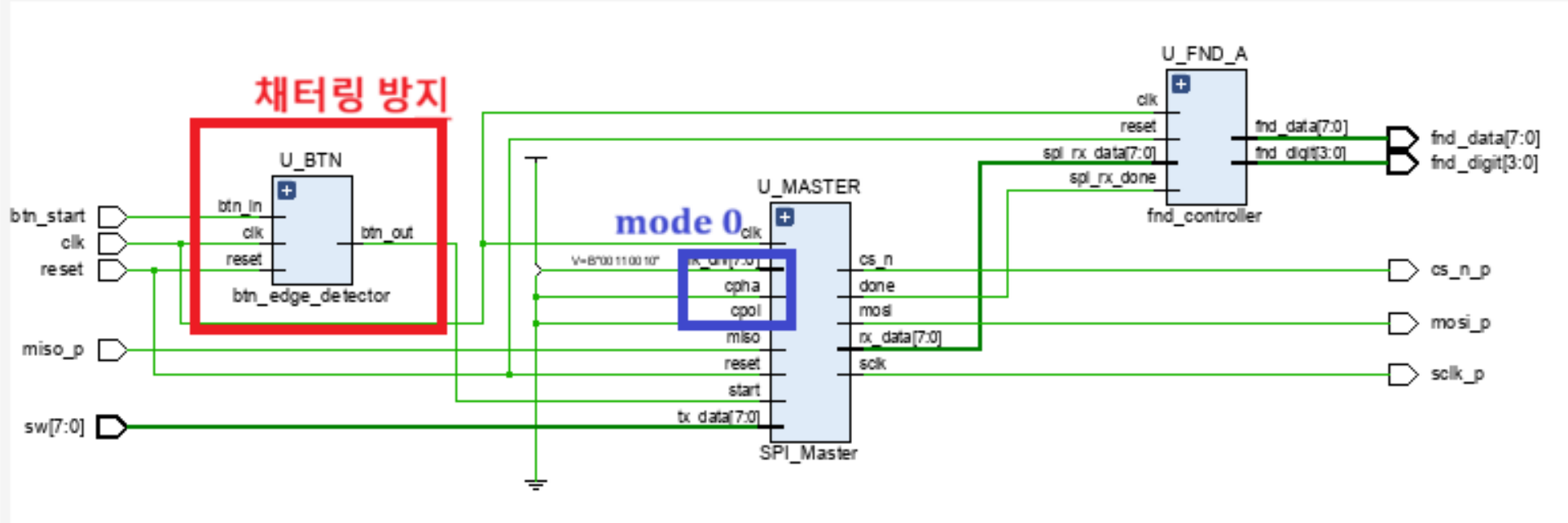
8비트 데이터의 양방향 송수신 및 카운팅이 완료되면 CS_n을 High로 복구시키고 통신 사이클을 정상적으로 종료.

```
if (!cpha) begin
    rx_shift_reg <= {rx_shift_reg[6:0], miso};
end else begin
    mosi <= tx_shift_reg[7];
    tx_shift_reg <= {tx_shift_reg[6:0], 1'b0};
end
```

```
    if (bit_cnt < 7) begin
        mosi <= tx_shift_reg[7];
        tx_shift_reg <= {tx_shift_reg[6:0], 1'b0};
    end
end else begin
    rx_shift_reg <= {rx_shift_reg[6:0], miso};
end
```

데이터를 읽거나(MISO) 데이터를 내보낼지(MOSI) 에 대한 DATA 송수신

SPI Master 모듈 설계 아키텍처



SPI Slave 모듈 설계

비동기 신호 처리

마스터에서 입력되는 SCLK, MOSI, CS_n 라인은 슬레이브의 내부 시스템 클럭과 완전히 독립적인 비동기(Asynchronous) 신호.

Metastability 방지: 각 통신 핀의 입력부에 2단 혹은 3단 D Flip-Flop Synchronizer 구조를 도입

```
always_ff @(posedge clk or posedge reset) begin
    if (reset) begin
        sclk_ff <= 3'b000;
        mosi_ff <= 2'b00;
        cs_n_ff <= 2'b11;
    end else begin
        sclk_ff <= {sclk_ff[1:0], sclk_i};
        mosi_ff <= {mosi_ff[0], mosi_i};
        cs_n_ff <= {cs_n_ff[0], cs_n_i};
    end
end
```

안전한 Edge Detection: 동기화된 플립플롭의 딜레이 값을 비교하여 클럭 엣지를 검출

```
wire sclk_rise = (sclk_ff[2] == 1'b0 && sclk_ff[1] == 1'b1);
```

SPI Slave 모듈 설계

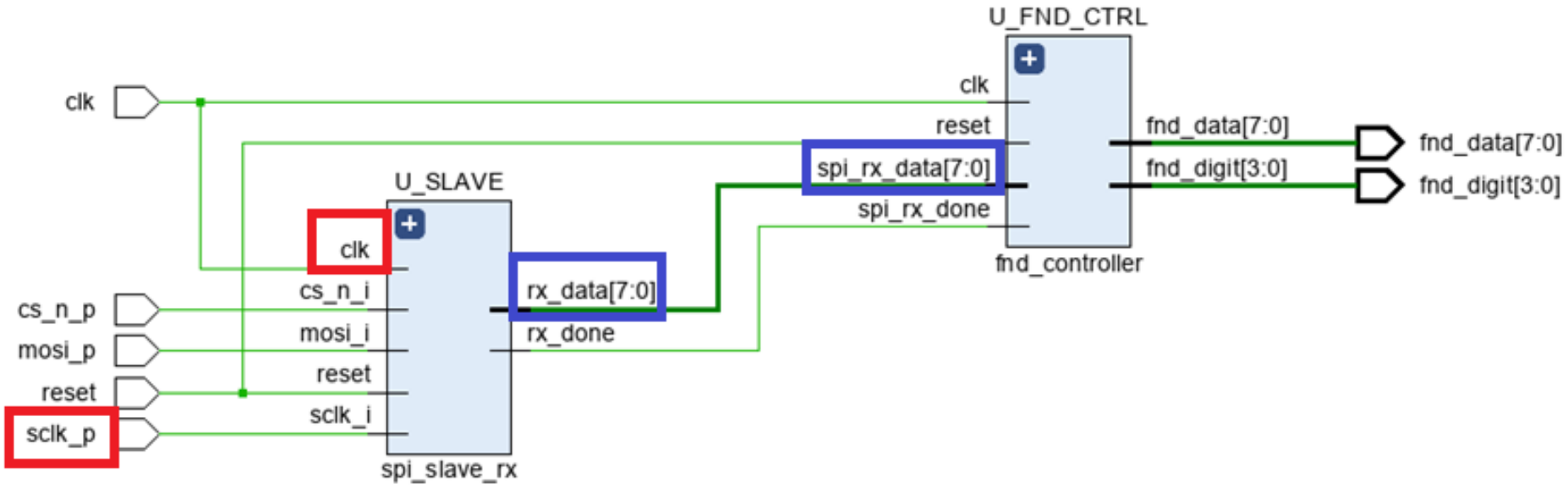
카운터 기반의 data shift SlaveFSM

CS_n 이 (1) 일때 전송할 데이터 로드

수신된 MOSI 비트를 LSB로 밀어넣고 shift

```
assign miso_o = shift_reg[7];
always_ff @(posedge clk or posedge reset) begin
    ~
    end else if (cs_n_ff[1]) begin
        ~
        shift_reg <= tx_data_i;
        bit_cnt <= 0;
    end else begin
        if (sclk rise) begin
            shift_reg <= {shift_reg[6:0], mosi_ff[1]};
            if (bit_cnt == 7) begin
                bit_cnt <= 0;
                rx_data <= {shift_reg[6:0], mosi_ff[1]};
                rx_done <= 1'b1;
            end else begin
                bit_cnt <= bit_cnt + 1;
            end
        end
    end
end
```

SPI Slave 모듈 설계



SPI 모듈 검증 - UVM 검증

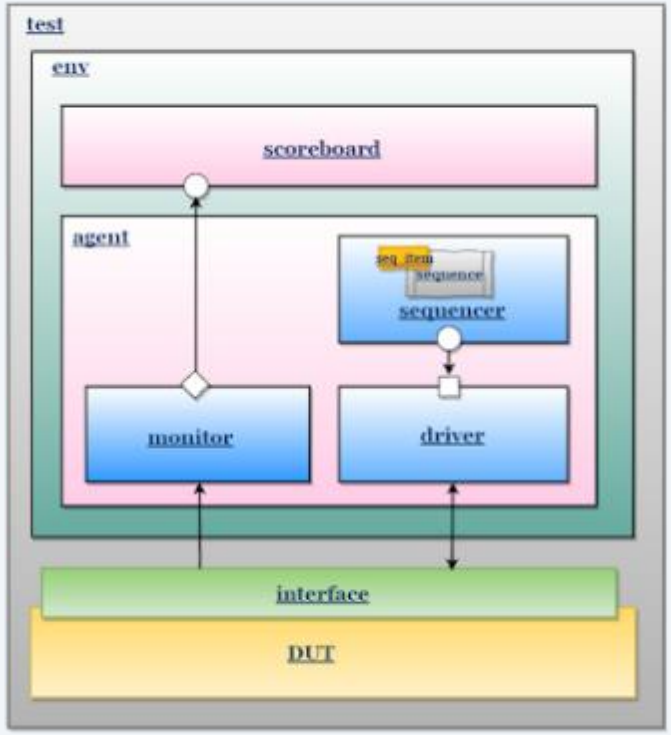
UVM 기반 자동화 테스트벤치 도입

SPI Master/Slave 독립적으로 검증

Sequence/Sequencer: All-Zero, All-one, 0x55, 0xAA 와 같은 코너 케이스(Corner Case) data를 자동 생성하여 통신 트랜잭션 주입.

Driver/Monitor: 가상의 Master 또는 Slave 역할을 수행하여 DUT에 정확한 타이밍으로 프로토콜 주입

Scoreboard: 송신 데이터와 수신 데이터가 일치하는지 비교하며 총 50번의 task 실행



SPI 모듈 검증 - UVM 검증

```
constraint c_data {
    tx_data dist {
        8'h00 :/ 1,    // All-Zeros (10%)
        8'hFF :/ 1,    // All-Ones  (10%)
        8'h55 :/ 1,    // 01010101  (10%)
        8'hAA :/ 1,    // 10101010  (10%)
        [8'h01:8'hFE] :/ 6 //      (60%)
    };
}
```

코너 케이스에 가중치 부여하여 검증 수행

Driver 로직

```
seq_item_port.get_next_item(req);
for (int i = 7; i >= 0; i--) begin
    vif.miso <= req.tx_data[i];
    @(posedge vif.sclk);
    req.rx_data[i] = vif.mosi;
    if (i > 0) @(posedge vif.sclk);
end
```

가상의 Slave 장치를 묘사하여 Master의 SCLK 타이밍에 맞춰 MISO 데이터 전송

SPI 모듈 검증 - UVM 검증

```
uvm_test_top [TEST] === [Iter 48] Master TX Start: 8'h55 ===
uvm_test_top.env.agt.drv [DRV] Slave MISO sent: 55  Master MOSI received: 55
uvm_test_top.env.scb [SCB_MATCH] MATCH Success!! Data = 0x55
uvm_test_top [TEST] === [Iter 49] Master TX Start: 8'h61 ===
uvm_test_top.env.agt.drv [DRV] Slave MISO sent: 63  Master MOSI received: 61
uvm_test_top.env.scb [SCB_MATCH] MATCH Success!! Data = 0x61
uvm_test_top [TEST] === [Iter 50] Master TX Start: 8'haa ===
uvm_test_top.env.agt.drv [DRV] Slave MISO sent: 55  Master MOSI received: aa
uvm_test_top.env.scb [SCB_MATCH] MATCH Success!! Data = 0xaa
```

랜덤하게 50번 수행

```
[spi_scoreboard] ==========================================
[spi_scoreboard]        SPI Master Scoreboard Summary
[spi_scoreboard] ==========================================
[spi_scoreboard]  Total Transactions : 50
[spi_scoreboard]  MATCH              : 50
[spi_scoreboard]  MISMATCH           : 0
[spi_scoreboard] ==========================================
[TEST_STATUS]  TEST PASSED!!!! 50 transactions complete
[spi_scoreboard]
```

Scoreboard 결과

SPI 모듈 검증 - UVM 검증

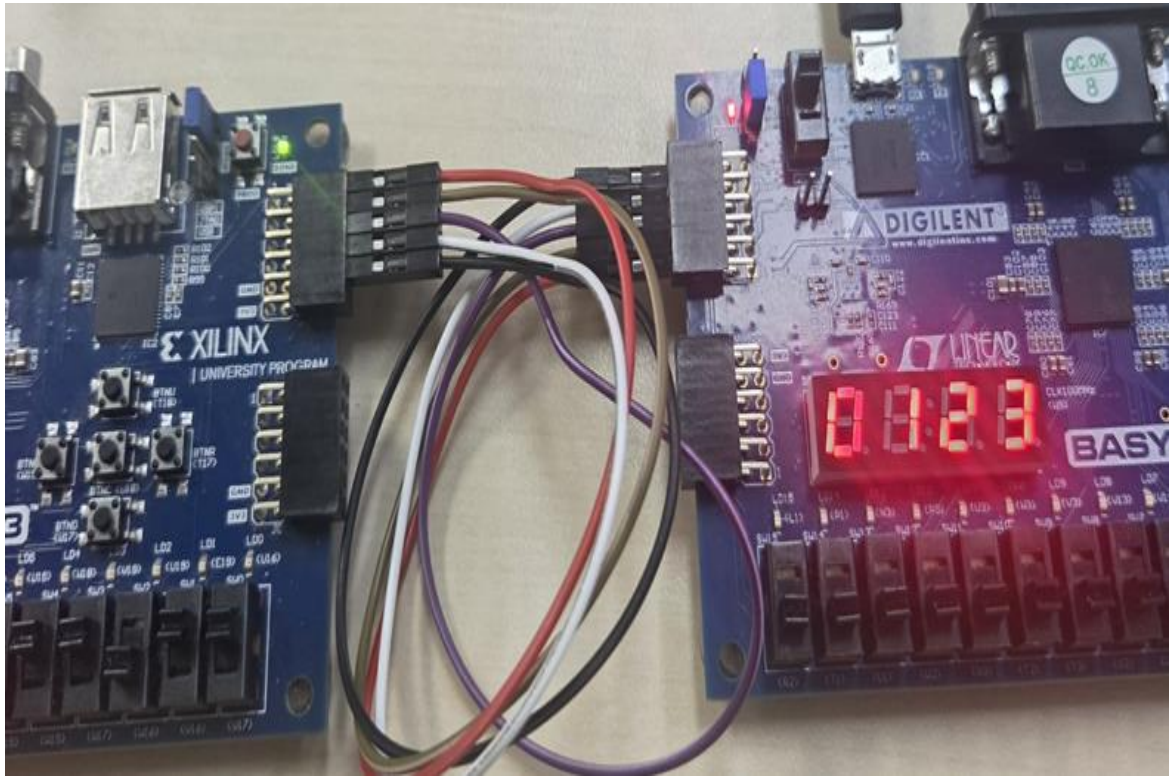
```
uvm_test_top.env.scb [SCB] Match Success!! Data=0x0e
uvm_test_top.env.agt.mon [MON] In =0xef, Out=0xef
uvm_test_top.env.scb [SCB] Match Success!! Data=0xef
uvm_test_top.env.agt.mon [MON] In =0xa1, Out=0xa1
uvm_test_top.env.scb [SCB] Match Success!! Data=0xa1
uvm_test_top.env.agt.mon [MON] In =0xa8, Out=0xa8
uvm_test_top.env.scb [SCB] Match Success!! Data=0xa8
uvm_test_top.env.agt.mon [MON] In =0xb1, Out=0xb1
uvm_test_top.env.scb [SCB] Match Success!! Data=0xb1
uvm_test_top.env.agt.mon [MON] In =0xb2, Out=0xb2
uvm_test_top.env.scb [SCB] Match Success!! Data=0xb2
uvm_test_top.env.agt.mon [MON] In =0x11, Out=0x11
uvm_test_top.env.scb [SCB] Match Success!! Data=0x11
```

랜덤하게 50번 수행

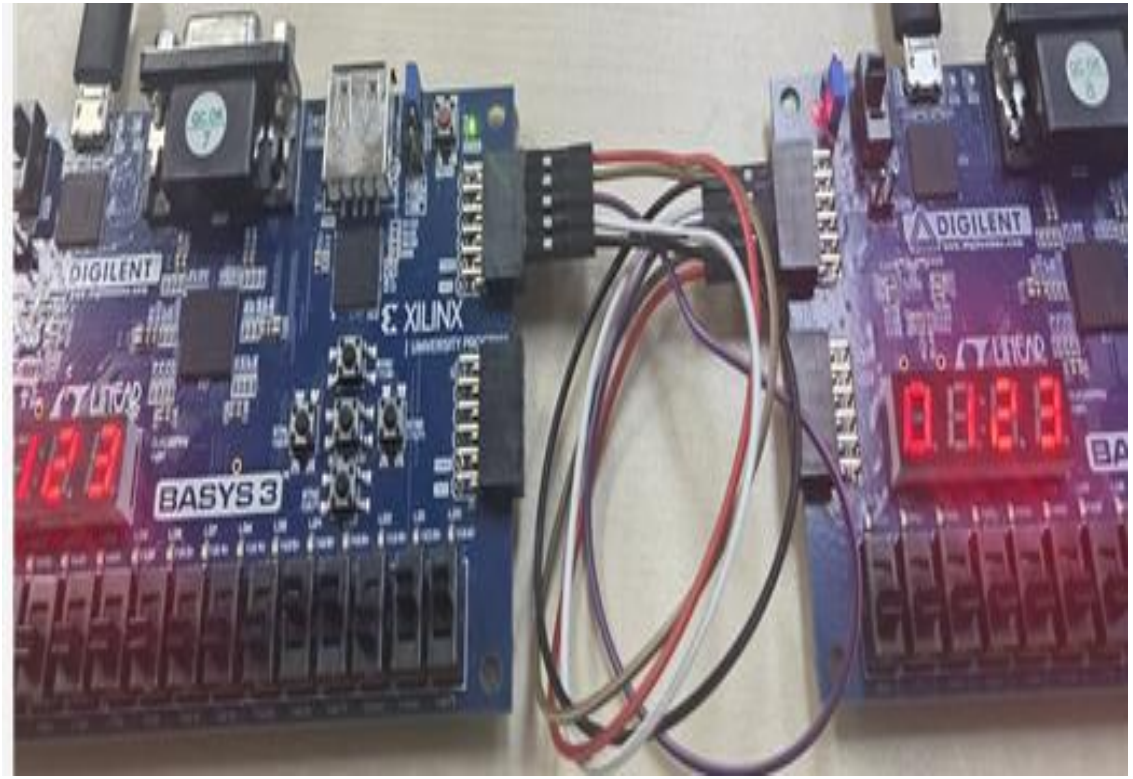
```
[slave_scoreboard] ==========================================
[slave_scoreboard]        SPI Slave Verification Summary
[slave_scoreboard] ==========================================
[slave_scoreboard]  Total Transactions : 50
[slave_scoreboard]  MATCH              : 50
[slave_scoreboard]  MISMATCH           : 0
[slave_scoreboard] ==========================================
[STATUS]  TEST PASSED
```

Scoreboard 결과

SPI 데모



Master -> Slave



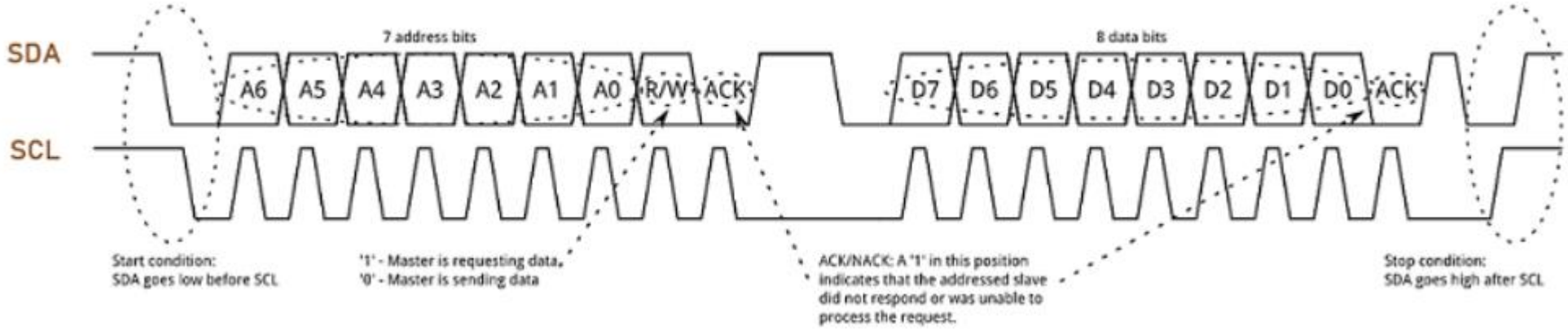
Slave -> Master

I2C 통신

I2C 통신의 기본 프레임 구조

I2C는 단 두 가닥의 선(SCL, SDA)만을 사용으로 데이터 선의 상태 변화로 통신의 시작과 끝을 제어

- Start : SCL이 High 상태일 때, SDA가 High에서 Low로 하강하는 순간 통신이 시작
- Stop : SCL이 High 상태일 때, SDA가 Low에서 High로 상승하며 버스 점유를 해제
- 데이터 샘플링: SCL이 High로 유지되는 구간 동안 SDA 데이터의 상태가 변경되지 않고 valid



I2C 모듈 설계

Master (Open-Drain)

sda_o가 1일 때 'Z' 를 출력하여 외부 pull up 저항이 전압을 올림

4단계 step(qtr_tick):
SCL의 한 주기를 4등분 하여 Start/Stop 조건 타이밍 확보
하강 -> SDA 변화 -> 상승 -> 유지

```
inout logic sda;
assign sda = sda_o ? 1'bz : 1'b0;
  ~
case (step)
    2'd0: scl_r <= 1'b0;
    2'd1: step  <= 2'd2;
    2'd2: scl_r <= 1'b1;
    2'd3: step  <= 2'd0;
endcase
~
```

I2C 모듈 설계

Slave (Ack 응답)

주소 일치 판별

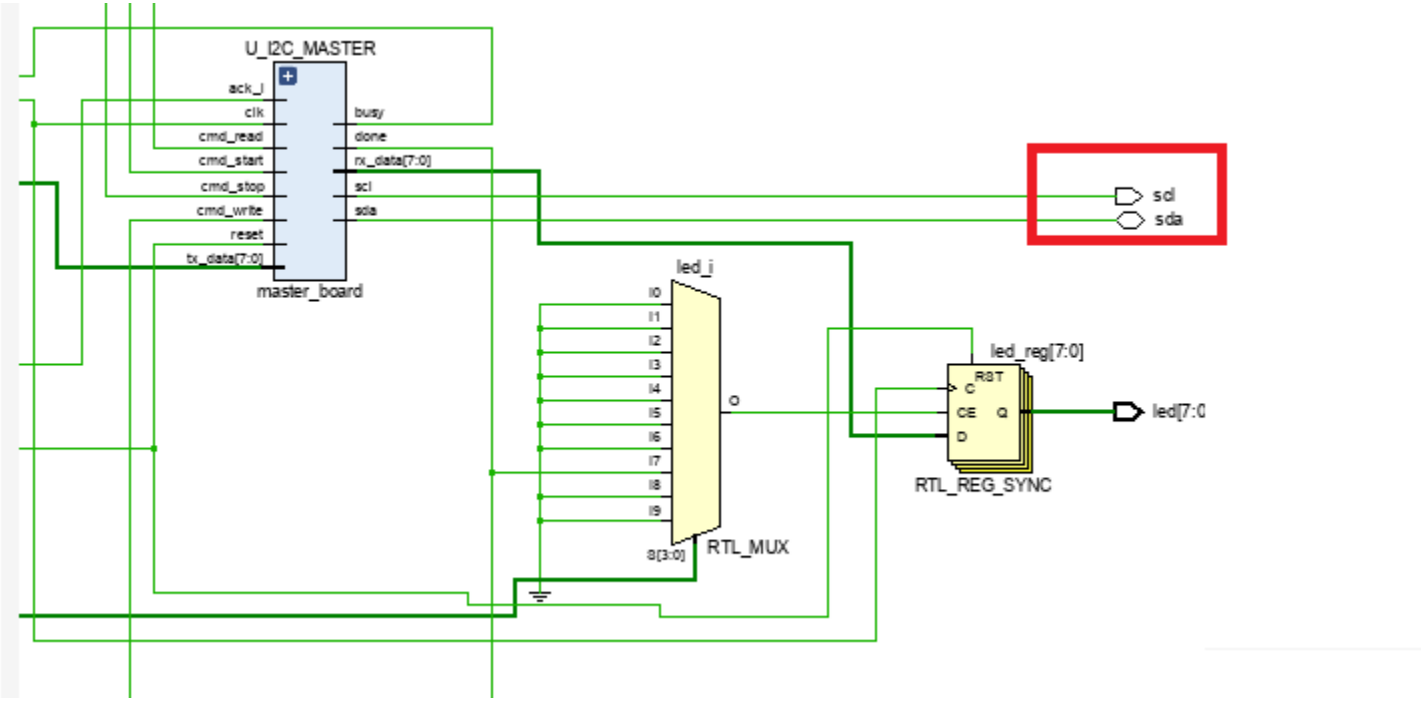
정해진 주소

```
if (bit_cnt == 4'd7) begin
    if (shift_reg[6:0] == SLAVE_ADDR) begin
        state <= (sda_in == 1'b1) ? R_DATA : W_DATA;
        match <= 1'b1;
    end
end
```

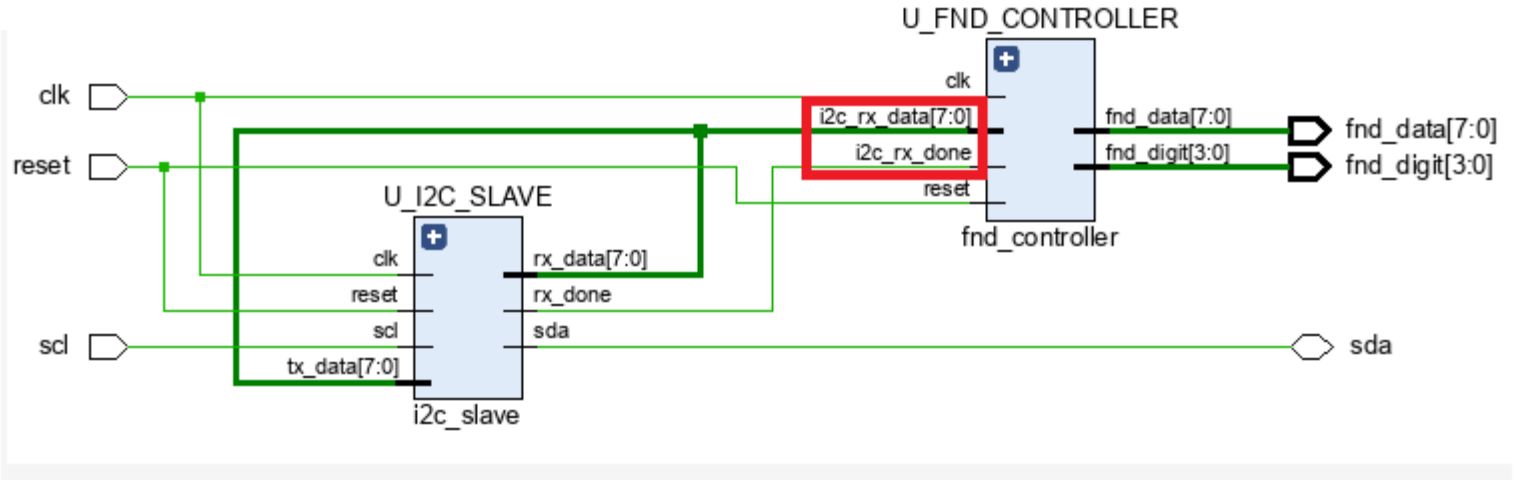
9번째 클럭에서 ACK 생성

```
ACK_W: begin
    if (scl_fall) begin
        state  <= IDLE;
        sda_en <= 0;
    end
end
```

I2C 모듈 설계


U_I2C_MASTER
ack_i
clk
cmd_read
cmd_start
cmd_stop
cmd_write
reset
tx_data[7:0]
busy
done
rx_data[7:0]
scl
sda
master_board
scl
sda
led_i
I0
I1
I2
I3
I4
I5
I6
I7
I8
I9
O
s[3:0]
RTL_MUX
led_reg[7:0]
RST
C
CE
D
Q
RTL_REG_SYNC
led[7:0

I2C 모듈 설계


U_FND_CONTROLLER
clk
i2c_rx_data[7:0]
i2c_rx_done
reset
fnd_data[7:0]
fnd_digit[3:0]
fnd_controller
fnd_data[7:0]
fnd_digit[3:0]
clk
reset
U_I2C_SLAVE
clk
reset
scl
tx_data[7:0]
rx_data[7:0]
rx_done
sda
i2c_slave
scl
sda

I2C 모듈 검증 - UVM 검증 (SPI 와 같은 방식)

```
uvm_test_top [TEST] === [task 43] Master TX Start: 8'hac ===
uvm_test_top.env.scb [SCB_MATCH] Match Success! Data=0xac
uvm_test_top [TEST] === [task 44] Master TX Start: 8'hd3 ===
uvm_test_top.env.scb [SCB_MATCH] Match Success! Data=0xd3
uvm_test_top [TEST] === [task 45] Master TX Start: 8'h41 ===
uvm_test_top.env.scb [SCB_MATCH] Match Success! Data=0x41
uvm_test_top [TEST] === [task 46] Master TX Start: 8'h59 ===
uvm_test_top.env.scb [SCB_MATCH] Match Success! Data=0x59
uvm_test_top [TEST] === [task 47] Master TX Start: 8'hfb ===
uvm_test_top.env.scb [SCB_MATCH] Match Success! Data=0xfb
uvm_test_top [TEST] === [task 48] Master TX Start: 8'h79 ===
uvm_test_top.env.scb [SCB_MATCH] Match Success! Data=0x79
uvm_test_top [TEST] === [task 49] Master TX Start: 8'h10 ===
uvm_test_top.env.scb [SCB_MATCH] Match Success! Data=0x10
uvm_test_top [TEST] === [task 50] Master TX Start: 8'h41 ===
uvm_test_top.env.scb [SCB_MATCH] Match Success! Data=0x41
```

랜덤하게 50번 수행

```
[i2c_scoreboard] ==========================================
[i2c_scoreboard]        I2C Master Scoreboard Summary
[i2c_scoreboard] ==========================================
[i2c_scoreboard]  MATCH    : 50
[i2c_scoreboard]  MISMATCH : 0
[STATUS] TEST PASSED!!!!
catcher.svh(705) @ 10001225000: reporter [UVM/REPORT/CATCHER]
```

Scoreboard 결과

I2C 모듈 검증 - UVM 검증

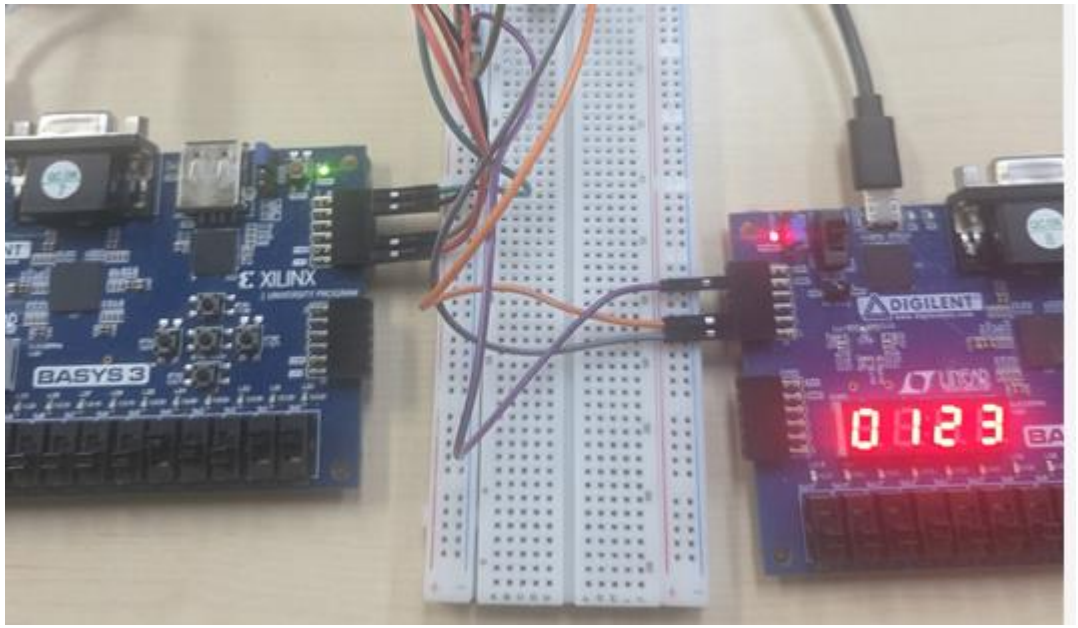
```
[SCB_MATCH] Match Success! Data=0x39
[SCB_MATCH] Match Success! Data=0xbc
[SCB_MATCH] Match Success! Data=0x99
[SCB_MATCH] Match Success! Data=0x73
[SCB_MATCH] Match Success! Data=0xb3
[SCB_MATCH] Match Success! Data=0xaa
[SCB_MATCH] Match Success! Data=0xfd
[SCB_MATCH] Match Success! Data=0xb4
[SCB_MATCH] Match Success! Data=0xeb
[SCB_MATCH] Match Success! Data=0xf9
[SCB_MATCH] Match Success! Data=0x1d
tion.svh(1276) @ 456200000: reporter [TEST DONE]
```

랜덤하게 50번 수행

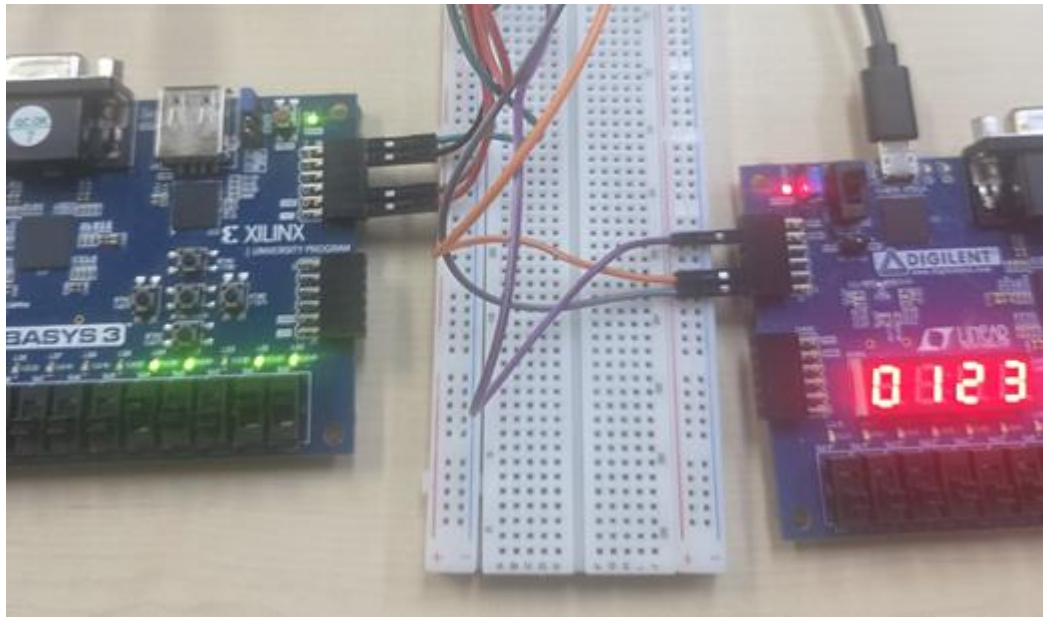
```
[i2c_slave_scoreboard] ========================================
[i2c_slave_scoreboard]        I2C Master Scoreboard Summary
[i2c_slave_scoreboard] ========================================
[i2c_slave_scoreboard]  MATCH    : 50
[i2c_slave_scoreboard]  MISMATCH : 0
[STATUS] TEST PASSED!!!!
rt_catcher.svh(705) @ 456200000: reporter [UVM/REPORT/CATCHER]
```

Scoreboard 결과

I2C 데모



Master -> Slave



Slave -> Master

Trouble Shooting

비동기 클럭 도메인 교차 (CDC) 문제

문제 상황: 마스터의 SCLK를 슬레이브 내부 클럭으로 직접 연결 시,타이밍 위반으로 인한 값 mismatch 발생

원인 분석: 두 모듈 간 클럭의 주파수와 위상이 전혀 다른 **비동기(Asynchronous) 상태**에서 발생하는 Metastability

해결: 슬레이브 SCLK/MOSI 입력단에 **D-FlipFlop Synchronizer**를 설계하여 내부 클럭으로 안전하게 동기화
플립플롭 간의 비교를 통해 클럭 엣지를 안정적으로 검출 -> Edge Detection 구조

결론 및 느낀 점

SPI 및 I2C 통신 모듈을 RTL로 직접 설계하고 통합해보며 비동기 클럭(CDC) 동기화나 open-drain 구조 등 하드웨어 고유의 물리적 문제들을 해결하고 이를 산업 표준인 UVM 으로 자동화 검증까지 수행하는 좋은 경험이었습니다.